Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

THM - Sweettooth Inc.

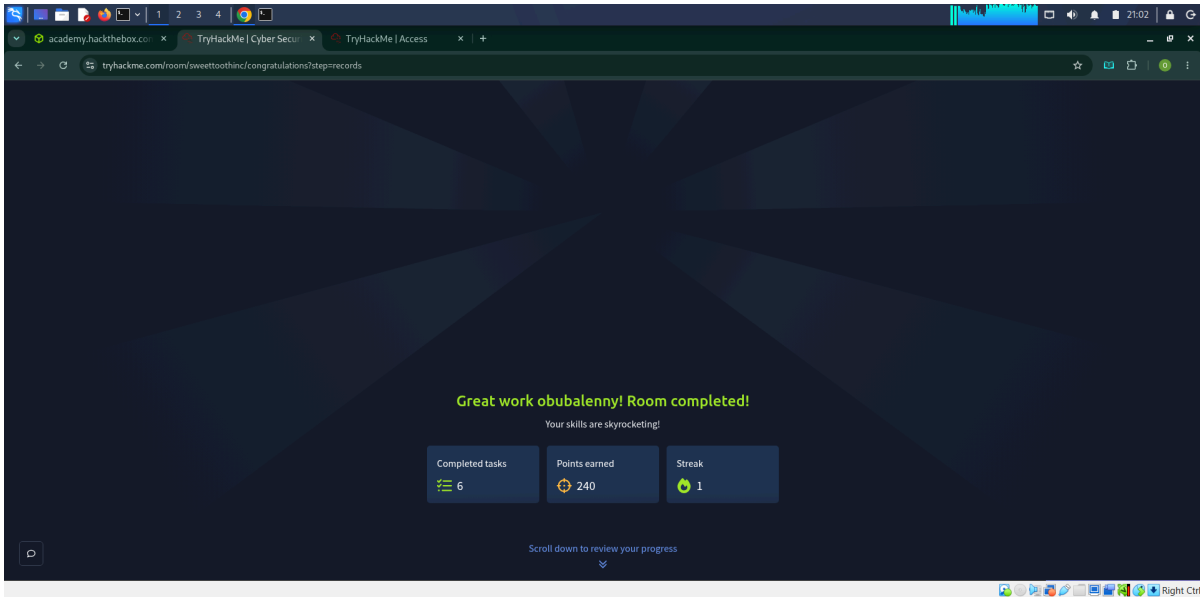
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 8, Assignment 2: THM - Sweettooth Inc.*
Platform	*TryHackMe*
Date submitted	*31/10/026*

1. Module completion

Shareable link

https://tryhackme.com/room/sweettoothinc/congratulations?step=room-completion-share

Completion screenshot



2. Introduction

Sweettooth Inc is a TryHackMe room centered on a misconfigured IoT-style monitoring stack: an InfluxDB time-series database exposed without properly secured authentication, sitting inside a Docker container with a dangerously overexposed Docker socket. The module walks through enumerating a service most security training doesn't cover in depth, exploiting a real-world authentication bypass class (a JWT signed with an empty shared secret), and following that access through to a full container escape. This matters to a security analyst because the vulnerabilities here aren't exotic, default or forgotten configuration settings on infrastructure services are exactly the kind of thing that shows up in real incident reports, and recognizing the pattern (a check that technically runs but was built on an assumption no one verified) matters more than memorizing any one CVE. Going in, I expected to practice basic service enumeration; what I got instead was a full chain from unauthenticated data exposure to host-level compromise, which gave me a much clearer picture of how a single weak link early in a system can cascade into everything after it.

3. The work

3.1 Enumeration

What this section covered

This section covered identifying the services exposed on the target and determining what each one was running. The goal was to map the attack surface before attempting any exploitation.

Question

Do a TCP portscan. What is the name of the database software running on one of these ports?

Answer

InfluxDB

How you got there

I started with a full TCP port scan across all 65535 ports to avoid missing anything hidden outside nmap's default top-1000:

nmap -p- --min-rate 5000 -oN allports.txt 10.49.178.179

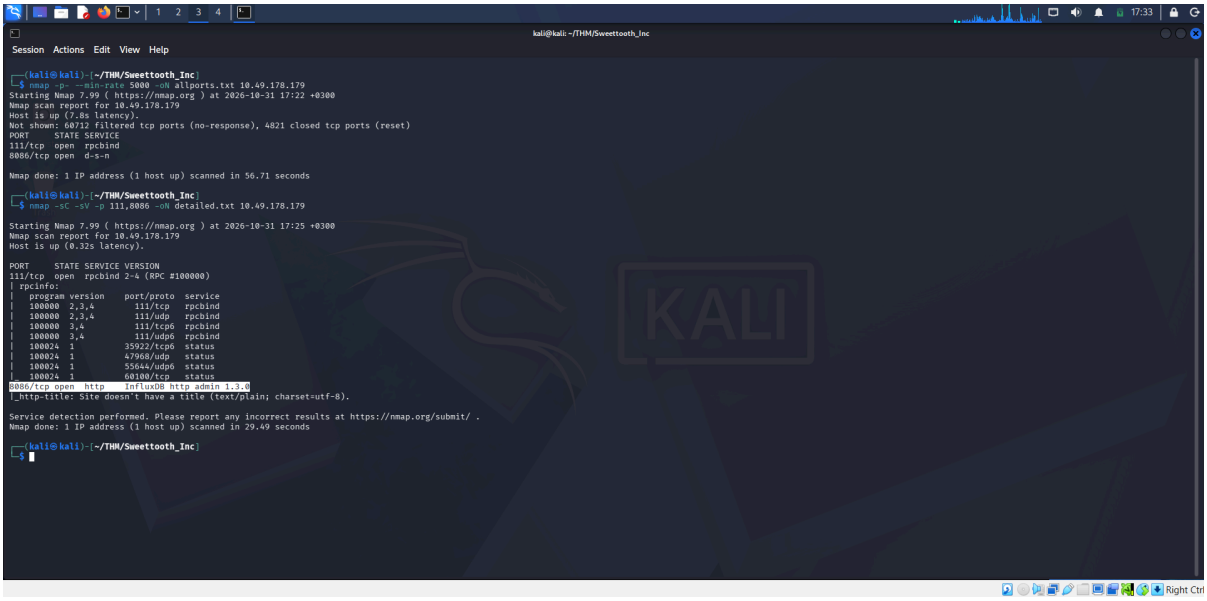
This returned two open ports: 111/tcp and 8086/tcp. I then ran a targeted service/version scan against just those two ports:

nmap -sC -sV -p 111,8086 -oN detailed.txt 10.49.178.179

Port 111 was identified as rpcbind, port 8086 was identified as InfluxDB http admin 1.3.0, directly naming the database software and its version from the HTTP service banner

Evidence

This screenshot shows the full and targeted nmap scans identifying ports 111 and 8086, with 8086 fingerprinted as InfluxDB 1.3.0.



It proves the database software and version were identified directly from the service scan.

3.2 Database exploration and user flag

What this section covered

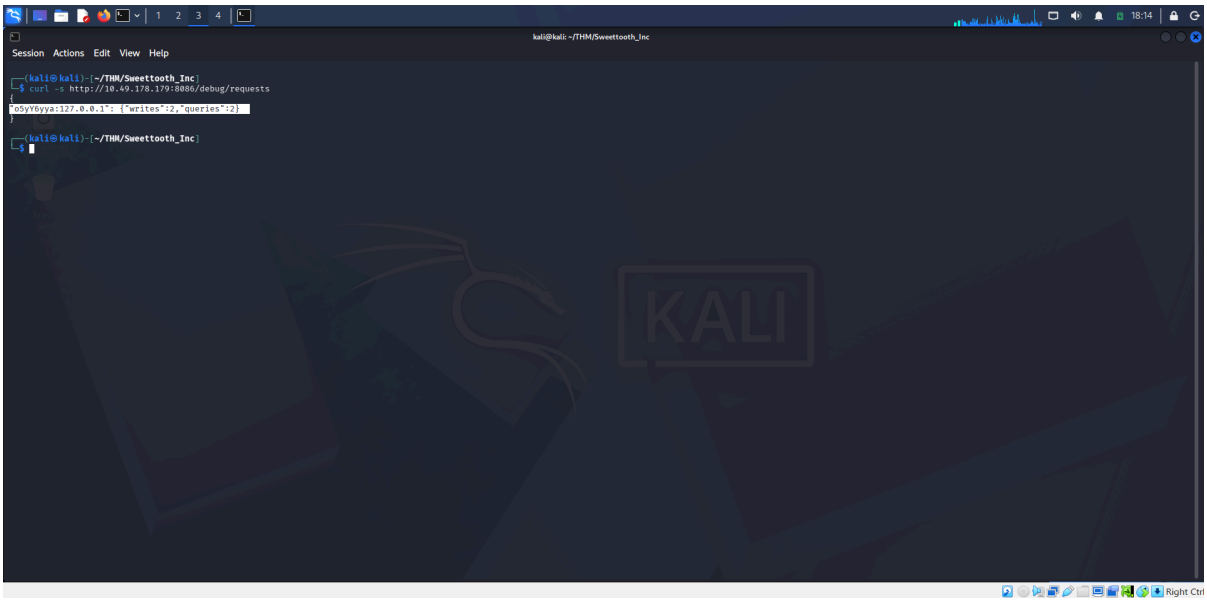
This section covered gaining actual read access to the InfluxDB instance identified during enumeration, then using that access to answer the room's sensor-data questions and locate credentials that led to a foothold on the underlying container.

Question 1

What is the database user you find?

Answer

o5yY6yya

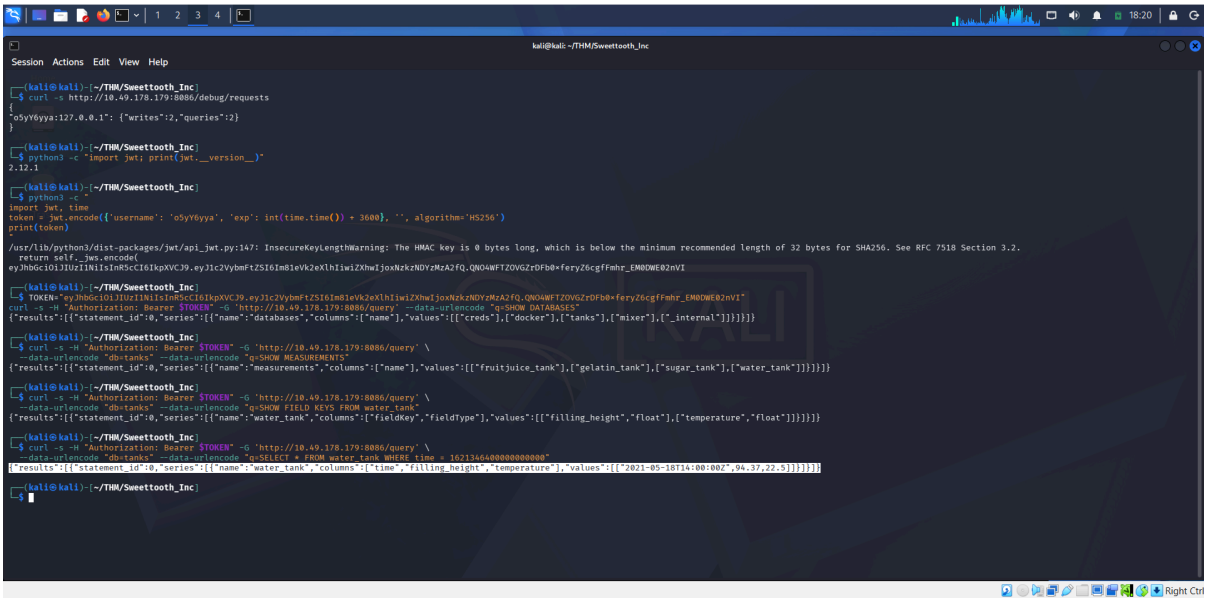


Question 2

What was the temperature of the water tank at 1621346400 (UTC Unix Timestamp)?

Answer

22.5

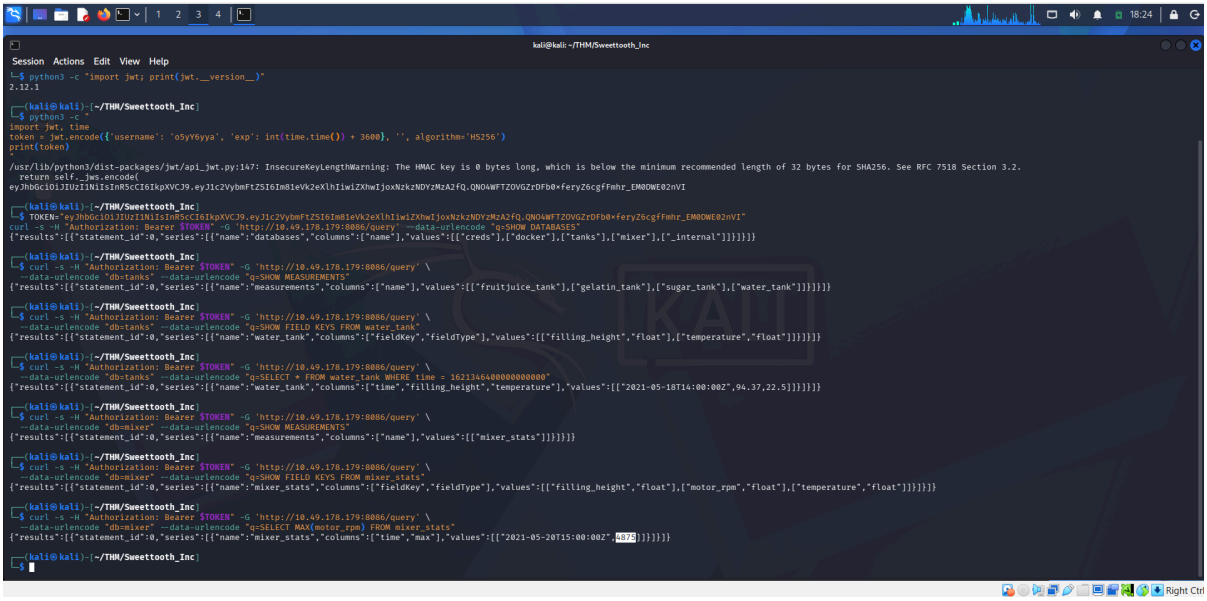


Question 3

What is the highest rpm the motor of the mixer reached?

Answer

4875

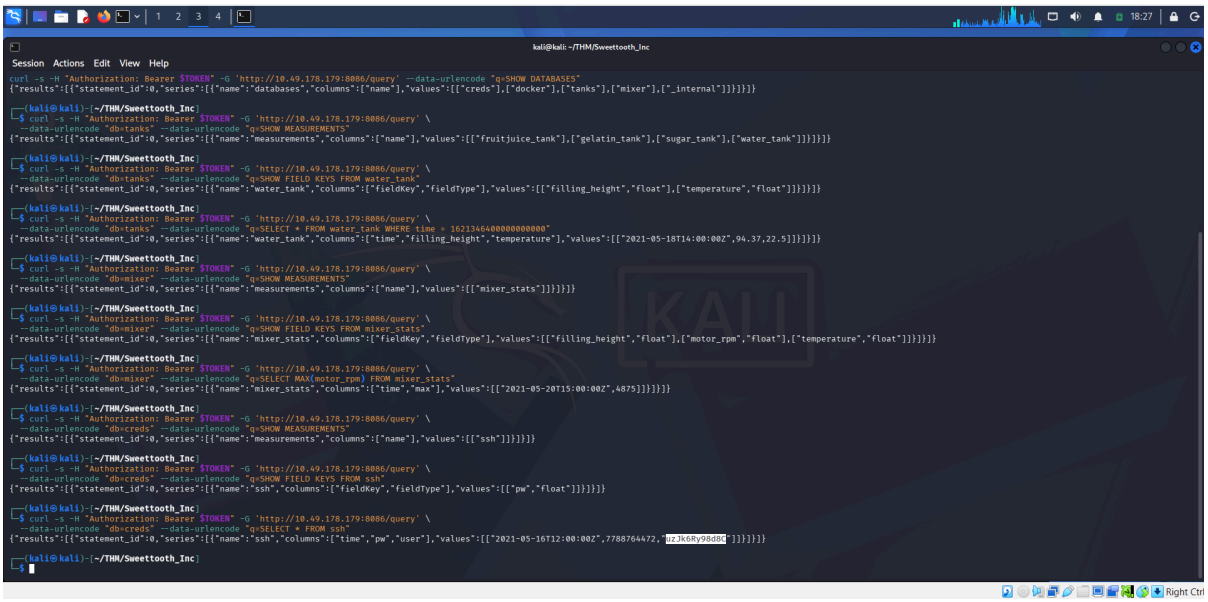


Question 4

What username do you find in one of the databases?

Answer

uzJk6Ry98d8C

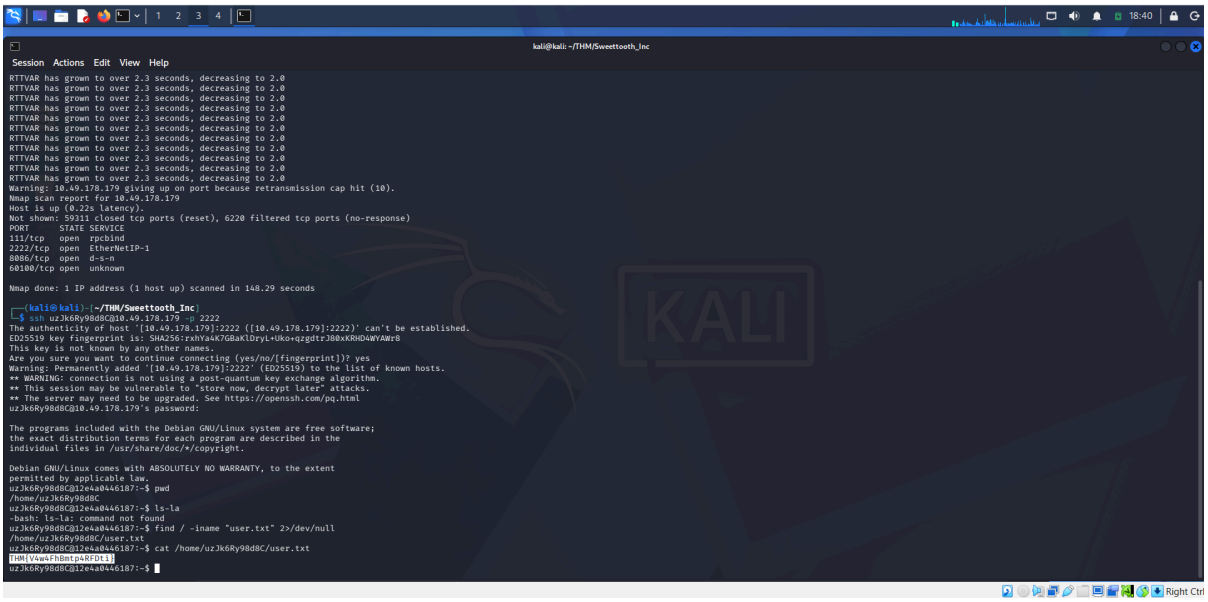


Question 5

user.txt

Answer

THM{V4w4FhBmtp4RFDti}



How you got there

I first queried InfluxDB directly (SHOW DATABASES) and got "unable to parse authentication credentials", confirming auth was enabled. A handful of default credential pairs and a short hydra run against common passwords both failed, and checking port 111 for NFS shares and a UDP sweep ruled out those as entry points too. Rather than keep guessing passwords, I looked at how the auth worked: InfluxDB 1.3.0 accepts JWT authentication, verifying each token's signature against a "shared secret", and when that secret is never explicitly configured, it defaults to empty, so anyone can forge a validly-signed token themselves. I pulled a real username (o5yY6yya) from the unauthenticated /debug/requests endpoint, forged a JWT for it signed with an empty key, and used it as a Bearer token, SHOW DATABASES then returned real results, confirming the bypass worked.

From there I explored each exposed database: tanks gave the water tank's temperature at the requested timestamp (22.5), mixer gave the motor's peak RPM (4875), and creds held SSH credentials for user uzJk6Ry98d8C. An initial SSH attempt on port 22 was refused, and a fresh full port scan showed why the service was actually listening on 2222, not 22. Connecting there with the discovered credentials landed a shell on a Docker container (sweettoothinc), where find / -iname "user.txt" located the flag at /home/uzJk6Ry98d8C/user.txt.

3.3 Privilege escalation

What this section covered

This section covered escalating from the low-privileged user obtained via SSH to root, by abusing an exposed and world-writable Docker socket to gain host-level control through the Docker API.

Question

/root/root.txt

Answer

THM{5qsDivHdCi2oabwp}

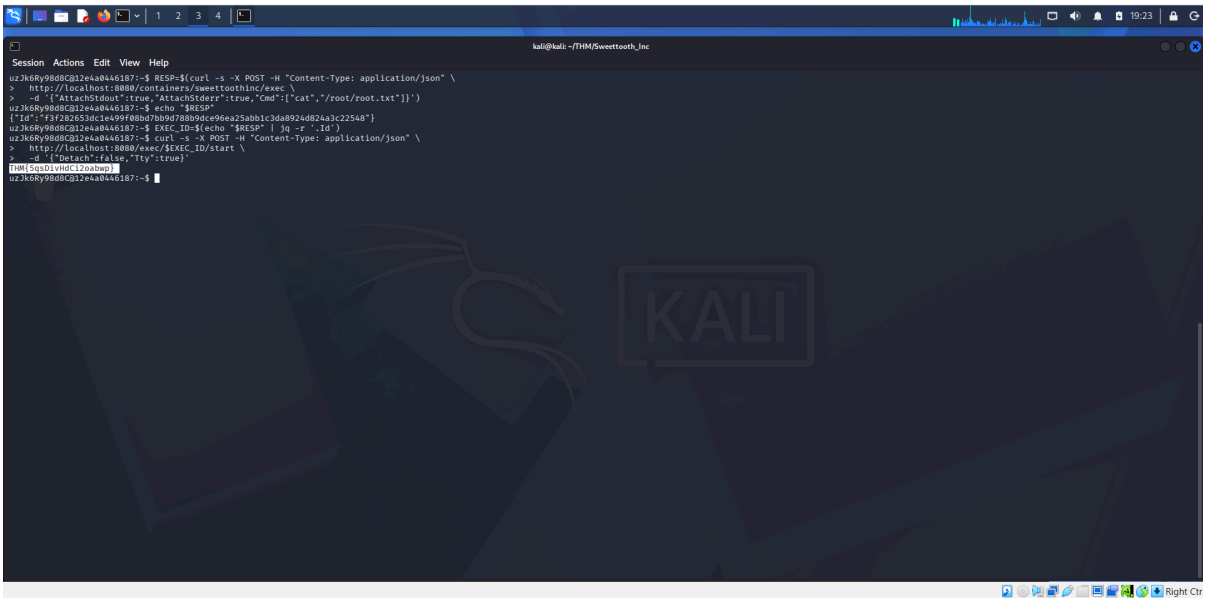
How you got there

With a shell as uzJk6Ry98d8C, I checked standard privilege-escalation vectors like sudo, SUID binaries, and cron jobs, but found nothing exploitable. The breakthrough came from ps aux, which revealed that the container's root entrypoint executed chmod a+rw /var/run/docker.sock at startup, making the Docker socket world-writable. Because access to this socket is equivalent to root privileges, and a socat process was already forwarding local TCP port 8080 to it, I was able to interact with the daemon directly. Since the docker CLI was missing on the machine, I used curl to query the API, confirming control and discovering that the sweettoothinc container socket was bind-mounted directly from the real host.

Rather than spawning a new container, I leveraged the Docker API's exec endpoint to run commands inside the already-running sweettoothinc instance. Because the container image does not enforce a low-privilege user, commands executed through this endpoint run as root by default. I sent a POST request to /containers/sweettoothinc/exec containing the command to read the flag, which generated an execution ID. Passing this ID into the /exec/<id>/start endpoint triggered the command and successfully returned the root flag from /root/root.txt, confirming full system compromise.

Evidence

This screenshot shows the Docker exec call returning the correct root flag from inside the running container.



It proves root-level command execution inside the target container was achieved via the exposed Docker socket.

3.4 Escape

What this section covered

This section covered going beyond root access inside the container to providing actual access to the underlying host filesystem, the real container-escape step, as opposed to the privilege escalation in 3.3 which stayed inside the container's own namespace.

Question

The second /root/root.txt

Answer

THM{nY2ZahyFABAmjrnx}

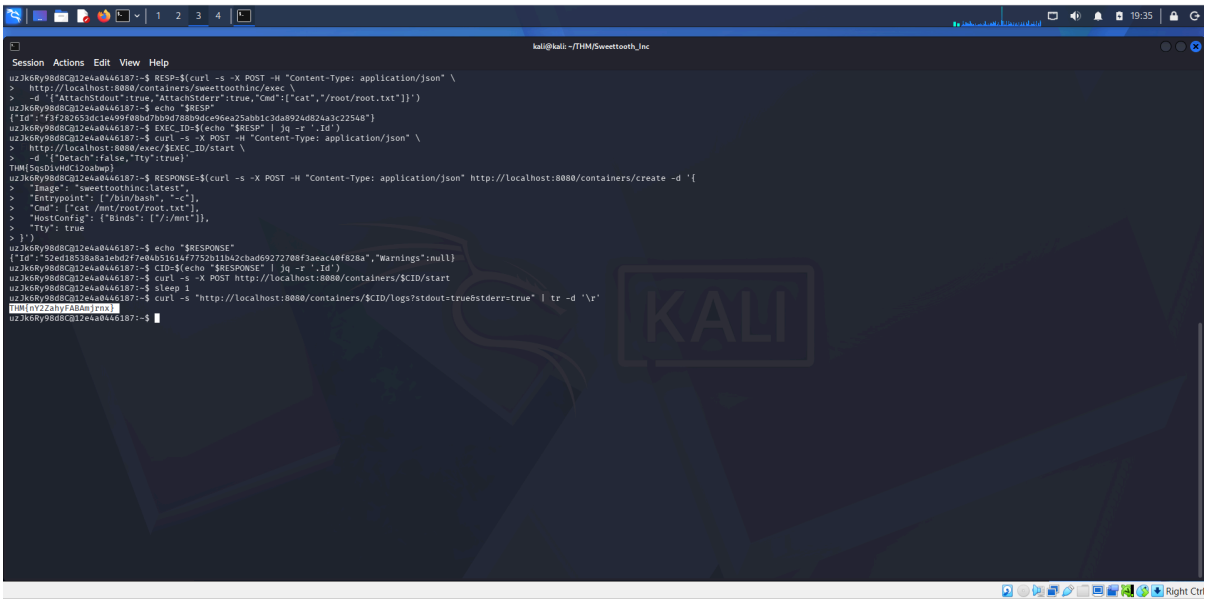
How you got there

To capture the second root flag, I exploited the writable Docker socket by creating a new container configured to break out of the isolated environment. Since the socket was bind-mounted from the true host, any container I deployed through it could also mount the host's filesystem. Using curl to interact with the Docker API's /containers/create endpoint, I instructed the daemon to spin up a new instance of sweettoothinc:latest and mount the entire host root directory (/) directly into the container's /mnt directory.

With the malicious container configured to run cat /mnt/root/root.txt upon startup, I extracted its container ID from the JSON response using jq. I then sent a follow-up POST request to the /start endpoint to launch the container, followed by a request to the /logs endpoint to capture the standard output. This successfully read the true host's root file through the volume mount, returning the final flag: THM{nY2ZahyFABAmjrnx}.

Evidence

This screenshot shows the Docker API request completing and returning the host-level root flag, read from /mnt/root/root.txt after bind-mounting the host's filesystem into a new container.



It proves access to the host filesystem was achieved through the Docker socket, confirming a true escape beyond the container's own boundary.

4. Challenges and how you resolved them

The biggest time sink during this lab was assuming the InfluxDB service could be breached via weak credentials. I spent considerable time running manual default credential checks, a hydra bruteforce against fasttrack.txt, and a scoped rockyou.txt dictionary attack, all of which came back empty alongside dead-end checks on the RPC service. However, systematically ruling out these traditional avenues was ultimately beneficial. It forced me to pivot away from guesswork and analyze the inner workings of InfluxDB's authentication mechanism, which ultimately led to uncovering the empty-secret JWT bypass vulnerability.

I also initially miscalculated the privilege escalation path for the container itself. My first exploit involved creating a new container to mount the host filesystem and read root.txt directly. While this aggressive approach worked, I realized it actually bypassed the intermediate objective of Task 4 and was meant for the final container breakout ("Escape") task instead. Recognizing this structural detour, I corrected my methodology to leverage a simpler docker exec command into the already-running sweettoothinc container, allowing me to properly secure root privileges inside the container environment first before executing the full host escape.

5. Conclusion

Going into this room, I could run a port scan and recognize common services, but I had no real sense of how a misconfiguration turns into root access, this room made that concrete, start to finish. What I

learned the most was how little of the chain depended on anything exotic: no custom exploit code, just configuration decisions nobody had revisited, from an unset authentication secret to a Docker socket made world-writable at container startup. If I did this again, I'd test how an authentication mechanism works before spending time brute-forcing it, since that would have saved the better part of an hour here. The single configuration decision that made this machine exploitable was the container's entrypoint script running chmod a+rw /var/run/docker.sock, handing world-readable and world-writable access to a socket that is functionally equivalent to root on the host. It should never have touched that socket's permissions at all; if a container genuinely needs to control Docker, that access belongs behind a narrowly-scoped proxy or API, not a bare chmod a+rw on the real daemon's socket. That is exactly the kind of overlooked setting a SOC analyst needs to be able to spot in a configuration review, long before it ever becomes an incident to respond to.